TUTORIAL

Single-Agent Navigation with Multiple Obstacles

Goal Attraction + Nearest-Obstacle Sigmoid Repulsion

Python / NumPy / Matplotlib

Educational objective

Understand how a mobile agent can navigate toward one goal while detecting multiple obstacles, selecting the nearest detected obstacle, and combining a goal-attraction force with a sigmoid obstacle-repulsion force.

1. What This Program Does

This simulation models one mobile agent moving in a two-dimensional environment. The agent starts at a known position, moves toward a single goal, senses the environment in 360 directions, and avoids multiple circular obstacles.

At every simulation step, the program computes two forces: a goal-attraction force that pulls the agent toward the goal, and an obstacle-repulsion force produced only by the nearest obstacle that is currently inside the sensing region.

Core navigation rule: Total force = Goal-attraction force + Repulsive force from the nearest detected obstacle.

Learning Objectives

- Define a 2D agent, goal, sensing radius, and multiple obstacles.
- Represent each obstacle with its own position, radius, sigmoid midpoint, sigmoid slope, and repulsion gain.
- Generate 360 sensing directions with 1-degree angular resolution.
- Use logistic sigmoid functions to shape attraction and repulsion behavior.
- Select the nearest obstacle inside the sensing range.
- Combine navigation forces and update the agent position using a discrete time step.
- Animate the agent, its path, sensing circle, and current obstacle detection.
- Plot the final navigation path after the simulation.

Required Python Packages

```
pip install numpy matplotlib
```

The program uses NumPy for vector mathematics and Matplotlib for the animation and path visualization.

2. Complete Program

The complete program is shown first. The following sections explain the same code step by step without changing its logic.

```
# -*- coding: utf-8 -*-
"""
Created on Mon Oct 12 22:53:21 2026

@author: engineer
"""


import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

plt.close("all")

# ============================================================
# 1. SIMULATION PARAMETERS
# ============================================================

dt = 0.1
num_iterations = 5000

# Goal attraction gain
Kg = 2.0

# Maximum velocity
MAX_SPEED = 5

# Distance used to determine that the goal was reached
GOAL_TOLERANCE = 5.0

# ============================================================
# 2. AGENT AND GOAL
# ============================================================

START = np.array([10.0, 10.0])
GOAL = np.array([90.0, 90.0])

# ============================================================
# 3. SENSOR PARAMETERS
# ============================================================

# Maximum sensing distance measured from the agent
SENSING_RADIUS = 22.0

# 360 sensing directions = 1 degree resolution
NUM_SENSING_DIRECTIONS = 360

# ============================================================
# 4. GOAL ATTRACTION SIGMOID
# ============================================================
```

```
GOAL_MID_DISTANCE = 12.0
GOAL_SLOPE = 0.25


# ============================================================
# 5. OBSTACLE LIST
# ============================================================
#
# Each obstacle has:
# name
# position
# physical radius
# sigmoid midpoint
# sigmoid slope
# repulsion gain
#
# ============================================================

OBSTACLES = [
  {
    "name": "Obstacle 1",
    "position": np.array([40.0, 34.0]),
    "radius": 6.0,
    "mid_distance": 10.0,
    "slope": 0.50,
    "gain": 3.5
  },
  {
    "name": "Obstacle 2",
    "position": np.array([65.0, 50.0]),
    "radius": 7.0,
    "mid_distance": 9.0,
    "slope": 0.60,
    "gain": 3.0
  },
  {
    "name": "Obstacle 3",
    "position": np.array([76.0, 70.0]),
    "radius": 5.0,
    "mid_distance": 8.0,
    "slope": 0.70,
    "gain": 2.8
  },
  {
    "name": "Obstacle 4",
    "position": np.array([40.0, 60.0]),
    "radius": 5.0,
    "mid_distance": 8.0,
    "slope": 0.70,
    "gain": 2
  },
  {
    "name": "Obstacle 5",
    "position": np.array([65.0, 80.0]),
    "radius": 2.0,
    "mid_distance": 8.0,
    "slope": 0.70,
    "gain": 1.8
  }
```

```
]

# ============================================================
# 6. CREATE 360 SENSING DIRECTIONS
# ============================================================

angles = np.linspace(0, 2 * np.pi, NUM_SENSING_DIRECTIONS, endpoint=False)
sensing_directions = np.column_stack((np.cos(angles), np.sin(angles)))

# ============================================================
# 7. GOAL SIGMOID
# ============================================================

def sigmoid_goal_attraction(distance):

    exponent = -GOAL_SLOPE * (distance - GOAL_MID_DISTANCE)
    exponent = np.clip(exponent, -60, 60)

    return 1.0 / (1.0 + np.exp(exponent))

# ============================================================
# 8. OBSTACLE REPULSION SIGMOID
# ============================================================

def sigmoid_obstacle_repulsion(distance, mid_distance, slope):
    """
    True decreasing logistic sigmoid.

    distance = mid_distance gives exactly:

        sigmoid = 0.5

    Close to obstacle:
        sigmoid -> 1

    Far from obstacle:
        sigmoid -> 0
    """

    exponent = slope * (distance - mid_distance)
    exponent = np.clip(exponent, -60, 60)

    return 1.0 / (1.0 + np.exp(exponent))

# ============================================================
# 9. GOAL ATTRACTION FORCE
# ============================================================

def calculate_goal_attraction(pos):

    vec_goal = GOAL - pos
    distance_goal = np.linalg.norm(vec_goal)
```

```
    if distance_goal < 0.0001:
        return np.zeros(2)

    direction_goal = vec_goal / distance_goal
    magnitude = sigmoid_goal_attraction(distance_goal)

    return direction_goal * magnitude

# ============================================================
# 10. FIND NEAREST DETECTED OBSTACLE
# ============================================================

def find_nearest_obstacle(pos):

    nearest_obstacle = None
    nearest_distance = np.inf

    for obstacle in OBSTACLES:

        vec = obstacle["position"] - pos
        distance_center = np.linalg.norm(vec)

        # Distance from agent to obstacle surface
        distance_surface = distance_center - obstacle["radius"]
        distance_surface = max(distance_surface, 0.0)

        # Check if obstacle is inside sensing region
        if distance_surface <= SENSING_RADIUS:

            if distance_surface < nearest_distance:
                nearest_distance = distance_surface
                nearest_obstacle = obstacle

    return nearest_obstacle, nearest_distance

# ============================================================
# 11. OBSTACLE REPULSION FORCE
# ============================================================

def calculate_obstacle_repulsion(pos):

    # ---------------------------------------------------------
    # Find nearest obstacle inside sensing range
    # ---------------------------------------------------------

    obstacle, distance_surface = find_nearest_obstacle(pos)

    # ---------------------------------------------------------
    # No obstacle detected
    # ---------------------------------------------------------

    if obstacle is None:
        return np.zeros(2), None, None, 0.0
```

```
    # ------------------------------------------------------------
    # Vector from agent to obstacle
    # ------------------------------------------------------------

    vec_obstacle = obstacle["position"] - pos
    distance_center = np.linalg.norm(vec_obstacle)

    if distance_center < 0.0001:
        return np.zeros(2), obstacle, 0.0, 0.0

    # ------------------------------------------------------------
    # Direction toward obstacle
    # ------------------------------------------------------------

    direction_to_obstacle = vec_obstacle / distance_center

    # ------------------------------------------------------------
    # Search among the 360 sensing directions
    # ------------------------------------------------------------

    similarity = sensing_directions @ direction_to_obstacle
    index = np.argmax(similarity)
    detected_direction = sensing_directions[index]

    # ------------------------------------------------------------
    # Sigmoid magnitude
    # ------------------------------------------------------------

    sigmoid_value = sigmoid_obstacle_repulsion(distance_surface, obstacle["mid_distance"], obstacle["slope"])

    # ------------------------------------------------------------
    # Repulsive force
    # ------------------------------------------------------------

    repulsion_force = -detected_direction * sigmoid_value * obstacle["gain"]

    return repulsion_force, obstacle, distance_surface, sigmoid_value

# ============================================================
# 12. SIMULATION
# ============================================================

position = START.copy()

positions_history = [position.copy()]
detected_obstacle_history = [None]
distance_obstacle_history = [None]
repulsion_history = [0.0]

for t in range(num_iterations):

    # ------------------------------------------------------------
```

```
# Goal attraction
# ------------------------------------------------------------

fg = Kg * calculate_goal_attraction(position)

# ------------------------------------------------------------
# Obstacle repulsion
# ------------------------------------------------------------

fo, detected_obstacle, distance_obstacle, sigmoid_value = calculate_obstacle_repulsion(position)

# ------------------------------------------------------------
# Total force
# ------------------------------------------------------------

force = fg + fo

# ------------------------------------------------------------
# Limit velocity
# ------------------------------------------------------------

speed = np.linalg.norm(force)

if speed > MAX_SPEED:
    force = force / speed * MAX_SPEED

# ------------------------------------------------------------
# Update position
# ------------------------------------------------------------

position = position + force * dt

# ------------------------------------------------------------
# Save history
# ------------------------------------------------------------

positions_history.append(position.copy())
detected_obstacle_history.append(detected_obstacle)
distance_obstacle_history.append(distance_obstacle)
repulsion_history.append(np.linalg.norm(fo))

# ------------------------------------------------------------
# Check goal
# ------------------------------------------------------------

distance_goal = np.linalg.norm(GOAL - position)

if distance_goal < GOAL_TOLERANCE:
    print()
    print("GOAL REACHED")
    print("Iteration:", t)
    print("Position:", position)
    break
```

```
positions_history = np.array(positions_history)

# ============================================================
# 13. ANIMATION
# ============================================================

fig, ax = plt.subplots(figsize=(9, 9))

ax.set_xlim(0, 100)
ax.set_ylim(0, 100)
ax.set_aspect("equal", adjustable="box")
ax.grid(True)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_title("Single Agent Navigation\nGoal Attraction + Nearest Obstacle Repulsion")

# ============================================================
# GOAL
# ============================================================

ax.scatter(GOAL[0], GOAL[1], marker="X", s=200, label="Goal")

# ============================================================
# START
# ============================================================

ax.scatter(START[0], START[1], marker="o", s=100, label="Start")

# ============================================================
# DRAW ALL OBSTACLES
# ============================================================

for obstacle in OBSTACLES:

    obstacle_circle = plt.Circle(obstacle["position"], obstacle["radius"], alpha=0.7)
    ax.add_patch(obstacle_circle)

    ax.text(
        obstacle["position"][0],
        obstacle["position"][1],
        obstacle["name"],
        horizontalalignment="center",
        verticalalignment="center"
    )

# ============================================================
# AGENT
# ============================================================

agent, = ax.plot([], [], "o", markersize=10, label="Agent")

# ============================================================
# PATH
# ============================================================
```

```
path_line, = ax.plot([], [], linewidth=2, label="Path")

# ============================================================
# SENSING RANGE
# ============================================================

sensing_circle = plt.Circle(START, SENSING_RADIUS, fill=False, linestyle="--", linewidth=1.5)
ax.add_patch(sensing_circle)

# ============================================================
# DETECTION LINE
# ============================================================

detection_line, = ax.plot([], [], linestyle="--", linewidth=1.2)

# ============================================================
# STATUS TEXT
# ============================================================

status_text = ax.text(2, 98, "", fontsize=10, verticalalignment="top")

ax.legend(loc="lower right")

# ============================================================
# 14. ANIMATION UPDATE
# ============================================================

def update(frame):

    pos = positions_history[frame]

    # ---------------------------------------------------------
    # Agent
    # ---------------------------------------------------------

    agent.set_data([pos[0]], [pos[1]])

    # ---------------------------------------------------------
    # Path
    # ---------------------------------------------------------

    path_line.set_data(positions_history[:frame + 1, 0], positions_history[:frame + 1, 1])

    # ---------------------------------------------------------
    # Move sensing range
    # ---------------------------------------------------------

    sensing_circle.center = (pos[0], pos[1])

    # ---------------------------------------------------------
    # Detected obstacle
    # ---------------------------------------------------------
```

```
    obstacle = detected_obstacle_history[frame]

    if obstacle is not None:

        detection_line.set_data(
            [pos[0], obstacle["position"][0]],
            [pos[1], obstacle["position"][1]]
        )

        status_text.set_text(
            "Obstacle detected: YES\n"
            f"Nearest: {obstacle['name']}\n"
            f"Distance: {distance_obstacle_history[frame]:.2f}\n"
            f"Repulsion force: {repulsion_history[frame]:.2f}"
        )

    else:

        detection_line.set_data([], [])

        status_text.set_text(
            "Obstacle detected: NO\n"
            "Nearest: ---\n"
            "Repulsion force: 0.00"
        )

    return agent, path_line, sensing_circle, detection_line, status_text

# ============================================================
# 15. ANIMATION
# ============================================================

ani = FuncAnimation(fig, update, frames=range(len(positions_history)), interval=20, repeat=True, repeat_delay=500)

# ============================================================
# 16. FINAL PATH
# ============================================================

fig2, ax2 = plt.subplots(figsize=(9, 9))

ax2.set_xlim(0, 100)
ax2.set_ylim(0, 100)
ax2.set_aspect("equal", adjustable="box")
ax2.grid(True)
ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.set_title("Final Navigation Path")

# ------------------------------------------------------------
# Complete path
# ------------------------------------------------------------

ax2.plot(positions_history[:, 0], positions_history[:, 1], linewidth=2, label="Agent Path")
```

```
# Start
ax2.scatter(START[0], START[1], s=100, label="Start")


# Goal
ax2.scatter(GOAL[0], GOAL[1], marker="X", s=200, label="Goal")


# Obstacles
for obstacle in OBSTACLES:

    obstacle_circle = plt.Circle(obstacle["position"], obstacle["radius"], alpha=0.7)
    ax2.add_patch(obstacle_circle)


    ax2.text(
        obstacle["position"][0],
        obstacle["position"][1],
        obstacle["name"],
        horizontalalignment="center",
        verticalalignment="center"
    )

ax2.legend()


plt.show()
```

3. Program Structure

The program is organized into sixteen numbered sections. The first sections define the environment and mathematical parameters. The middle sections calculate the attraction and repulsion forces. The final sections run the simulation and create the animation and final path plot.

4. Simulation Parameters

```
dt = 0.1
num_iterations = 5000
Kg = 2.0
MAX_SPEED = 5
GOAL_TOLERANCE = 5.0
```

dt is the simulation time step. At every iteration, the position is updated using the current force multiplied by dt. A smaller dt normally produces smoother motion but requires more iterations.

num_iterations is the maximum number of simulation steps. The loop may stop earlier if the agent reaches the goal.

Kg scales the goal-attraction force. MAX_SPEED limits the magnitude of the final motion command. GOAL_TOLERANCE defines how close the agent must be to the goal before the simulation declares success.

5. Agent Start and Goal

```
START = np.array([10.0, 10.0])
GOAL = np.array([90.0, 90.0])
```

The agent begins at coordinate (10, 10) and attempts to reach the single goal at coordinate (90, 90). Both are represented as two-element NumPy vectors containing X and Y coordinates.

6. 360-Degree Sensing

```
SENSING_RADIUS = 22.0
NUM_SENSING_DIRECTIONS = 360

angles = np.linspace(0, 2 * np.pi, NUM_SENSING_DIRECTIONS, endpoint=False)
sensing_directions = np.column_stack((np.cos(angles), np.sin(angles)))
```

The agent has a circular sensing region with radius 22 units. The 360 sensing directions provide one direction for every degree around the agent.

angles contains 360 angles from 0 to almost 2*pi radians. The cosine and sine of those angles are combined to create 360 unit vectors. These vectors represent the directions the agent can evaluate around itself.

Interpretation: 360 directions means approximately 1-degree angular resolution around the agent.

7. Goal-Attraction Sigmoid

```
GOAL_MID_DISTANCE = 12.0
GOAL_SLOPE = 0.25

def sigmoid_goal_attraction(distance):
    exponent = -GOAL_SLOPE * (distance - GOAL_MID_DISTANCE)
    exponent = np.clip(exponent, -60, 60)
    return 1.0 / (1.0 + np.exp(exponent))
```

The attraction magnitude is generated with a logistic sigmoid. When the agent is far from the goal, the sigmoid approaches 1. As the agent gets close to the goal, the attraction becomes smaller.

GOAL_MID_DISTANCE is the distance at which the sigmoid equals 0.5. GOAL_SLOPE controls how quickly the attraction changes around that midpoint.

np.clip limits the exponent to a safe numerical range and prevents overflow in the exponential calculation.

8. Defining Multiple Obstacles

The obstacles are stored in a Python list. Each entry is a dictionary containing all information needed to describe one obstacle and its repulsive behavior.

Field	Meaning
name	Text label used in the animation.
position	Obstacle center position [X, Y].
radius	Physical radius of the circular obstacle.
mid_distance	Distance where the obstacle sigmoid equals 0.5.
slope	Controls how sharply repulsion changes with distance.
gain	Scales the repulsive-force magnitude for that obstacle.

```
OBSTACLES = [
    {
        "name": "Obstacle 1",
        "position": np.array([40.0, 34.0]),
        "radius": 6.0,
        "mid_distance": 10.0,
        "slope": 0.50,
        "gain": 3.5
    },
    ...
]
```

Because every obstacle has its own midpoint, slope, and gain, each obstacle can generate a different repulsion profile even though all obstacles use the same mathematical sigmoid function.

9. Obstacle-Repulsion Sigmoid

```
def sigmoid_obstacle_repulsion(distance, mid_distance, slope):
    exponent = slope * (distance - mid_distance)
    exponent = np.clip(exponent, -60, 60)
    return 1.0 / (1.0 + np.exp(exponent))
```

This is a decreasing logistic sigmoid. When distance equals mid_distance, the output is exactly 0.5. When the agent is very close to the obstacle, the output approaches 1. When the agent is far away, the output approaches 0.

The function receives mid_distance and slope as arguments because those values are different for each obstacle.

10. Goal-Attraction Force

```
def calculate_goal_attraction(pos):
    vec_goal = GOAL - pos
    distance_goal = np.linalg.norm(vec_goal)

    if distance_goal < 0.0001:
        return np.zeros(2)

    direction_goal = vec_goal / distance_goal
    magnitude = sigmoid_goal_attraction(distance_goal)
    return direction_goal * magnitude
```

First, the vector from the current agent position to the goal is calculated. Its Euclidean norm gives the distance to the goal. Dividing the vector by the distance produces a unit vector that points toward the goal.

The unit direction is multiplied by the sigmoid magnitude. The result is a two-dimensional attraction-force vector.

11. Finding the Nearest Detected Obstacle

```
def find_nearest_obstacle(pos):
    nearest_obstacle = None
    nearest_distance = np.inf

    for obstacle in OBSTACLES:
        vec = obstacle["position"] - pos
        distance_center = np.linalg.norm(vec)
        distance_surface = distance_center - obstacle["radius"]
        distance_surface = max(distance_surface, 0.0)

        if distance_surface <= SENSING_RADIUS:
            if distance_surface < nearest_distance:
                nearest_distance = distance_surface
                nearest_obstacle = obstacle

    return nearest_obstacle, nearest_distance
```

The program checks every obstacle in the list. The distance is measured from the agent to the obstacle surface, not only to the center. This is important because obstacles can have different physical radii.

Only obstacles whose surface is within SENSING_RADIUS are considered detected. If several obstacles are inside the sensing region, the one with the smallest surface distance becomes the active obstacle for repulsion.

Important: Only the nearest detected obstacle contributes the repulsive force at a given simulation step. The forces from all obstacles are not summed.

12. Calculating the Repulsive Force

```
direction_to_obstacle = vec_obstacle / distance_center
similarity = sensing_directions @ direction_to_obstacle
index = np.argmax(similarity)
detected_direction = sensing_directions[index]

sigmoid_value = sigmoid_obstacle_repulsion(distance_surface, obstacle["mid_distance"], obstacle["slope"])
repulsion_force = -detected_direction * sigmoid_value * obstacle["gain"]
```

The normalized vector direction_to_obstacle points from the agent toward the selected obstacle. The matrix multiplication with sensing_directions computes the dot product between that direction and each of the 360 sensing vectors.

The largest dot product identifies the sensing direction that most closely points toward the obstacle. The negative of that direction points away from the obstacle, so it becomes the repulsion direction.

The sigmoid determines the distance-dependent strength, while obstacle["gain"] scales the final repulsive force.

13. Main Simulation Loop

```
for t in range(num_iterations):
    fg = Kg * calculate_goal_attraction(position)
    fo, detected_obstacle, distance_obstacle, sigmoid_value = calculate_obstacle_repulsion(position)
    force = fg + fo

    speed = np.linalg.norm(force)
    if speed > MAX_SPEED:
        force = force / speed * MAX_SPEED
```

```
    position = position + force * dt
```

Every iteration follows the same sequence: calculate goal attraction, calculate the repulsion from the nearest detected obstacle, add the two forces, limit the maximum magnitude, and update the position.

The position update uses a simple discrete Euler integration: new position = current position + force x dt. In this simulation, the force vector acts directly as a motion or velocity command.

14. Saving the Simulation History

```
positions_history.append(position.copy())
detected_obstacle_history.append(detected_obstacle)
distance_obstacle_history.append(distance_obstacle)
repulsion_history.append(np.linalg.norm(fo))
```

The simulation stores the position and detection information from every step. These arrays are later used by the animation. Saving copies of the positions is important because NumPy arrays are mutable.

15. Goal-Reached Condition

```
distance_goal = np.linalg.norm(GOAL - position)

if distance_goal < GOAL_TOLERANCE:
    print()
    print("GOAL REACHED")
    print("Iteration:", t)
    print("Position:", position)
    break
```

After each movement step, the program measures the distance from the agent to the goal. If that distance is less than GOAL_TOLERANCE, the simulation prints the final information and exits the loop.

16. Building the Animation

The animation window displays the goal, start point, all obstacles, the moving agent, the trajectory accumulated so far, the circular sensing range, and a dashed detection line toward the currently selected obstacle.

```
sensing_circle = plt.Circle(START, SENSING_RADIUS, fill=False, linestyle="--", linewidth=1.5)
detection_line, = ax.plot([], [], linestyle="--", linewidth=1.2)
```

The sensing circle is centered on the agent and moves with it. Its dashed boundary allows students to see exactly when an obstacle enters or leaves the sensing region.

17. Updating Each Animation Frame

```
pos = positions_history[frame]
agent.set_data([pos[0]], [pos[1]])
path_line.set_data(positions_history[:frame + 1, 0], positions_history[:frame + 1, 1])
sensing_circle.center = (pos[0], pos[1])
```

For every animation frame, the agent is moved to the stored position, the path is extended, and the sensing circle is recentered on the agent.

If an obstacle was detected at that frame, a dashed line is drawn between the agent and that obstacle. The status text also reports the obstacle name, distance, and repulsive-force magnitude.

18. Repeating the Animation

```
ani = FuncAnimation(fig, update, frames=range(len(positions_history)), interval=20, repeat=True,
```

```
repeat_delay=500)
```

repeat=True causes the animation to start again after the final frame. repeat_delay=500 introduces a 500-millisecond delay before the replay begins. The simulation itself is calculated once; the stored trajectory is what repeats.

19. Final Path Plot

The second figure shows the complete trajectory after the simulation. It displays the start, goal, all obstacles, and the final agent path. This static plot is useful for comparing trajectories after changing gains, sigmoid parameters, or obstacle locations.

20. Algorithm Summary

1. Initialize the agent at START.
2. Calculate the attraction vector toward GOAL.
3. Check all obstacles and identify which ones are inside SENSING_RADIUS.
4. Select the nearest detected obstacle based on distance to its surface.
5. Use the 360 sensing directions to identify the direction toward that obstacle.
6. Generate a sigmoid repulsive force in the opposite direction.
7. Add the goal-attraction and obstacle-repulsion forces.
8. Limit the resulting motion to MAX_SPEED.
9. Update the agent position using dt.
10. Store the data for animation and repeat until the goal is reached.

21. Student Experiments

Once the original program is working, students can modify one parameter at a time and observe how the trajectory changes.

Parameter	Try changing	Observe
SENSING_RADIUS	22 -> 12 or 30	How early the agent reacts to obstacles.
Kg	2.0 -> 1.0 or 4.0	How strongly the goal dominates navigation.
mid_distance	Change one obstacle only	Where its repulsion reaches 50%.
slope	0.70 -> 0.2 or 1.2	How gradual or abrupt the repulsion becomes.
gain	Increase one obstacle gain	How strongly that obstacle pushes the agent away.
radius	Change physical obstacle size	How surface distance changes obstacle selection.
dt	0.1 -> 0.05	Smoothness and numerical behavior of the trajectory.

22. Connection to the Physical Spider Robot

This simulation treats the agent as a point that can move directly in any X-Y direction. A physical Spider robot cannot instantly move in an arbitrary direction because it has an orientation and discrete motion commands.

The next implementation step is therefore to keep the same navigation-force calculation but convert the resulting desired force direction into a heading error. That heading error can then be mapped to the Spider commands already used in the physical platform: forward, turn in one direction, turn in the other direction, or stop.

Key idea: The potential-field logic can determine where the robot should move; the physical navigation

controller determines how the Spider turns and drives to follow that desired direction.

23. Key Takeaways

- Goal attraction continuously biases the agent toward one destination.
- Every obstacle can have its own repulsion gain and sigmoid shape.
- Only the nearest obstacle inside the sensing region produces repulsion.
- The 360 sensing directions provide a one-degree directional search.
- The moving dashed circle visualizes the sensing radius during navigation.
- The program stores the complete trajectory for animation and final path analysis, providing a direct foundation for obstacle-aware navigation on the physical Spider robot.

